Design Verification Based on UVM for the Systolic Array Design

TetraMem Confidential

All rights reserved

Background – Matrix Multiplication

Matrix multiplication is a major part of the computation for many applications. Here we consider a 3x3 matrix multiplication: C = AxB + D

For simplicity, for this lab we will ignore D and consider only C = AxB only. Eg:

```
for (int k = 0; k < 3; k++) {
  for (int j = 0; j < 3; j++) {
    for (int i = 0; i < 3; i++) {
        C[i,j] += A[i,k] * B[k,j];
        }
  }
}
```

TetraMem Confidential

All rights reserved

Background – Processing Elements

A Processing Element (PE) is a basic ALU unit that we will use in this systolic array implementations.

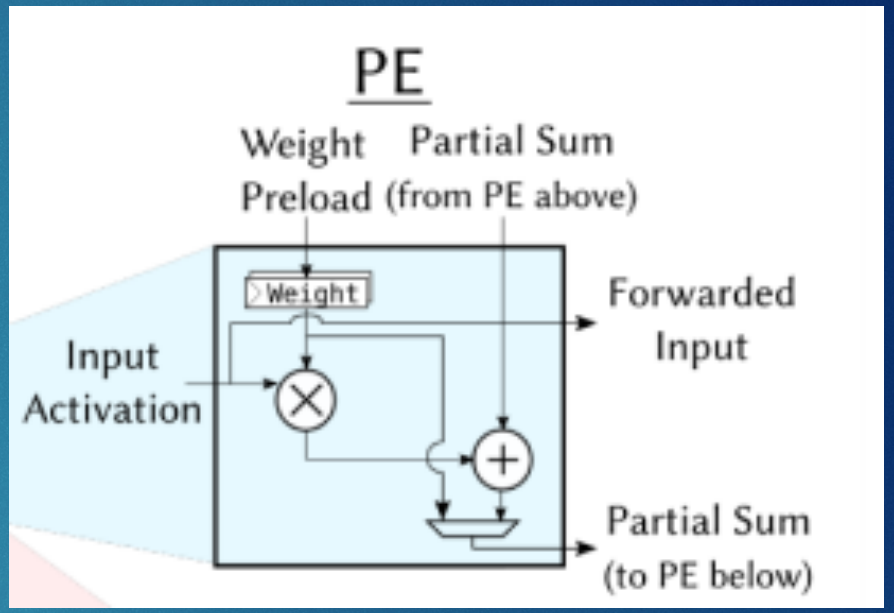
A basic PE has 3 data inputs, 3 data outputs.
Data Inputs:

- Input Activation (a)
- Weight Preload (b)
- Partial Sum from PE above (cin)

Data Outputs:

- Forwarded Input Activation (a) to the right
- Forwarded Weight (b) to below
- Partial Sum (cout = a*b + cin)

Other IOs to each PE (clk, rst_n, control signals etc) are not listed

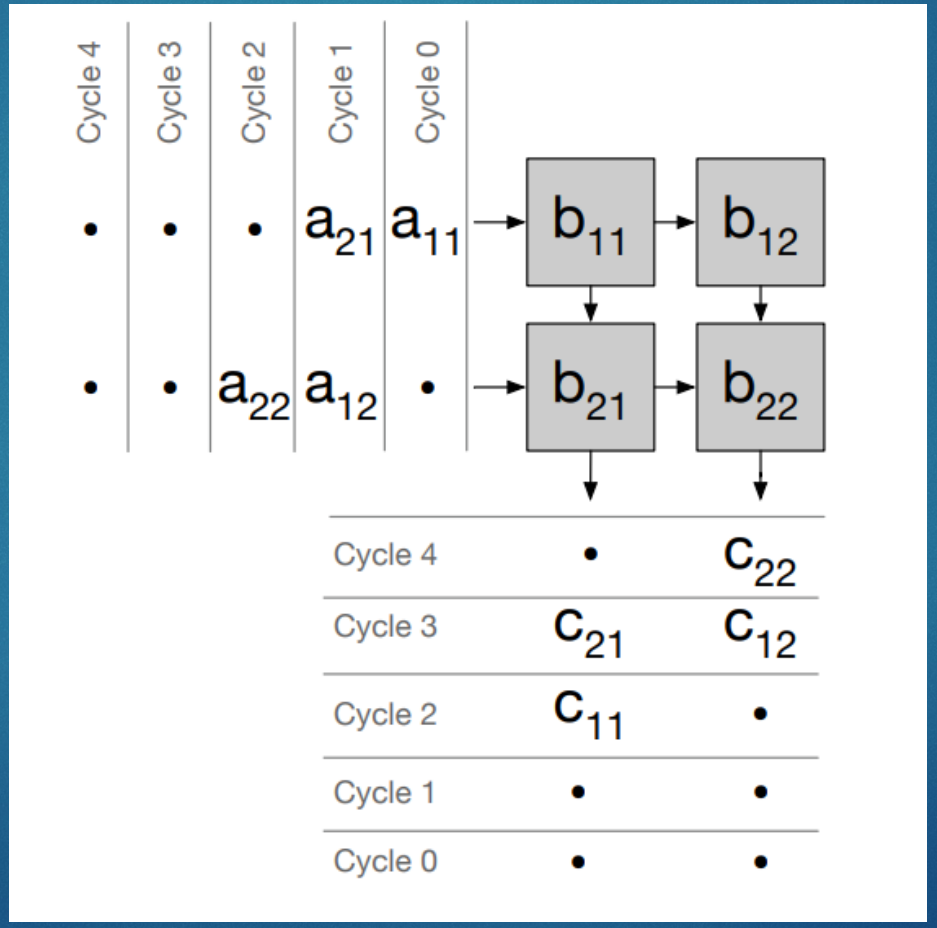


TetraMem Confidential

All rights reserved

Example of an 2x2 Systolic Array

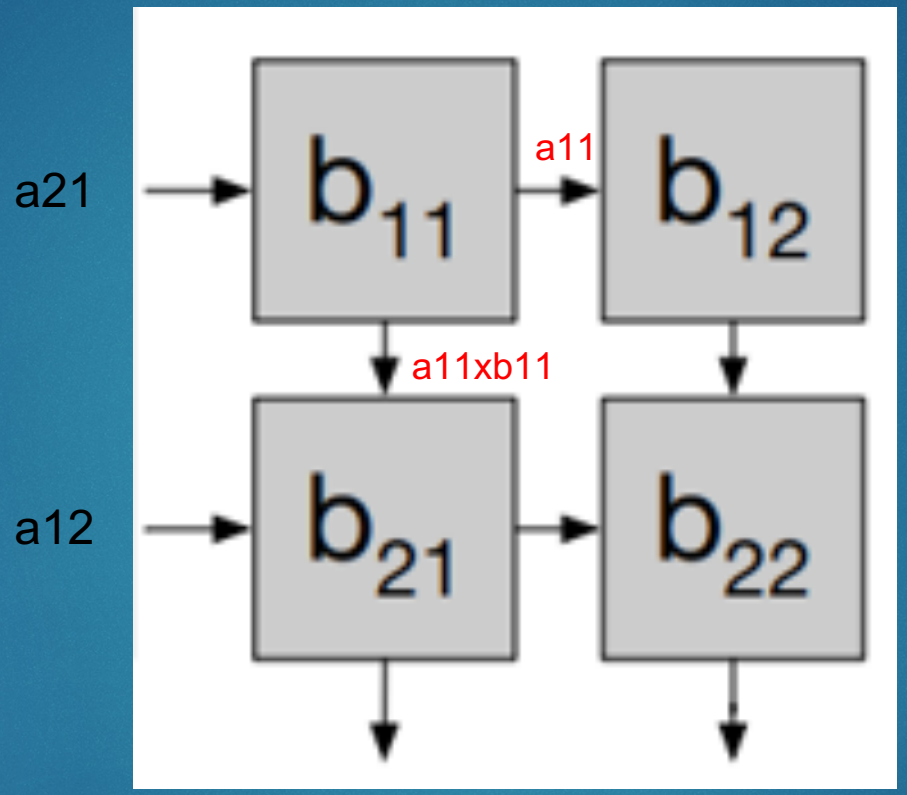
Let us first assume that B matrix is already loaded to the systolic array.
To compute C = AxB, we load A to the systolic array and read out the output C from the bottom.
Note that It takes 5 cycles to complete C = AxB for a 2x2 systolic array



TetraMem Confidential

All rights reserved

2x2 Systolic Array Example – cycle 1



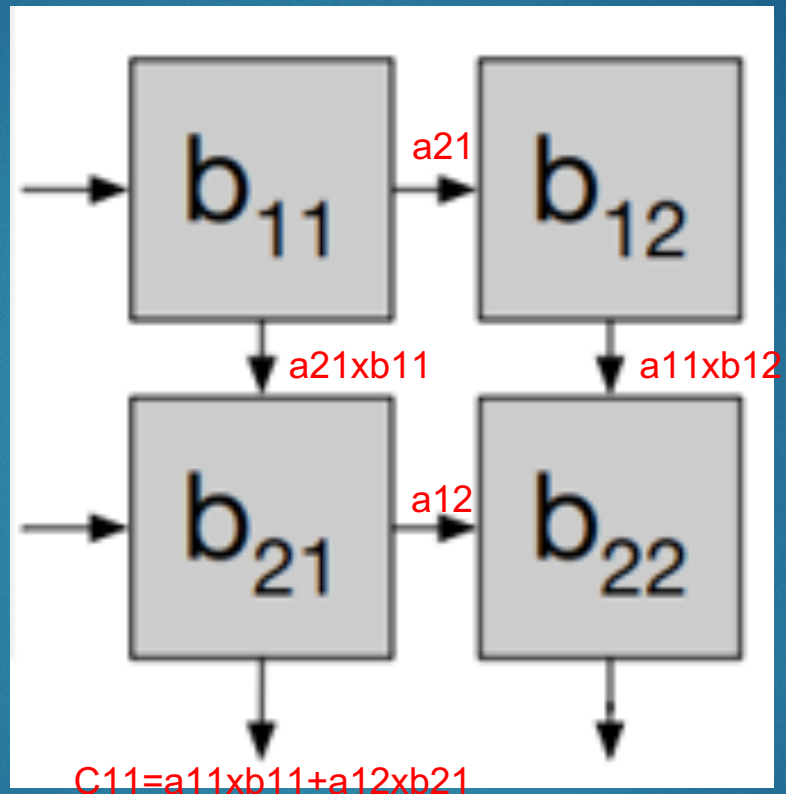
TetraMem Confidential

All rights reserved

2x2 Systolic Array Example – cycle 2

.

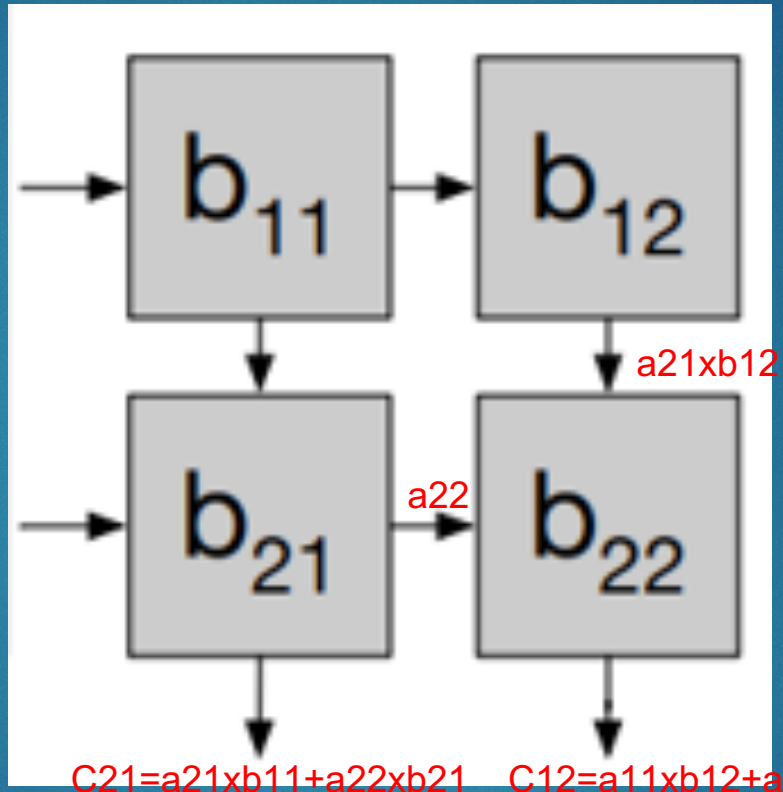
a22



TetraMem Confidential

All rights reserved

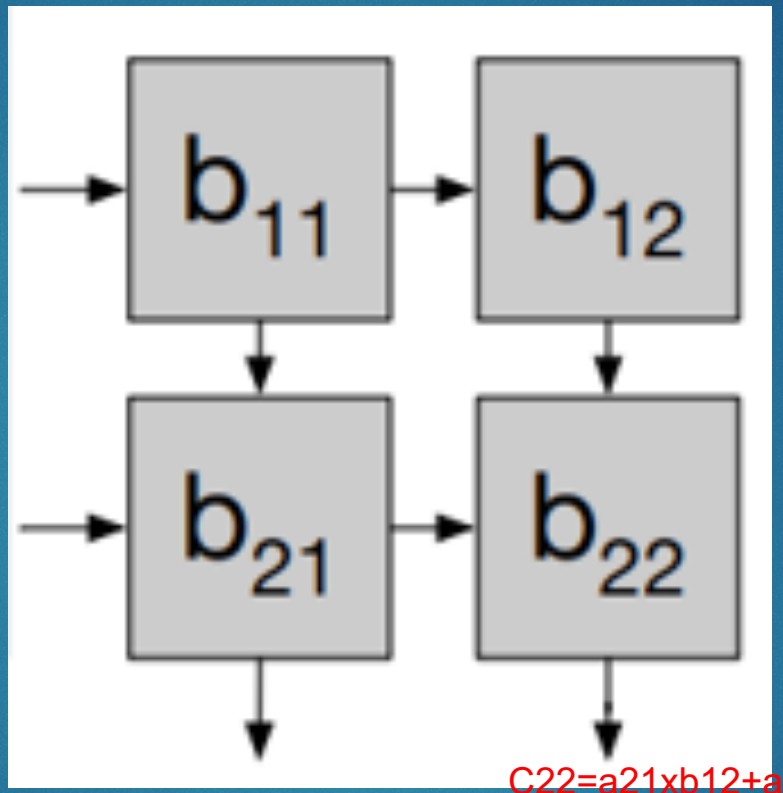
2x2 Systolic Array Example – cycle 3



TetraMem Confidential

All rights reserved

2x2 Systolic Array Example – cycle 4



C22=a21xb12+a22xb22

TetraMem Confidential

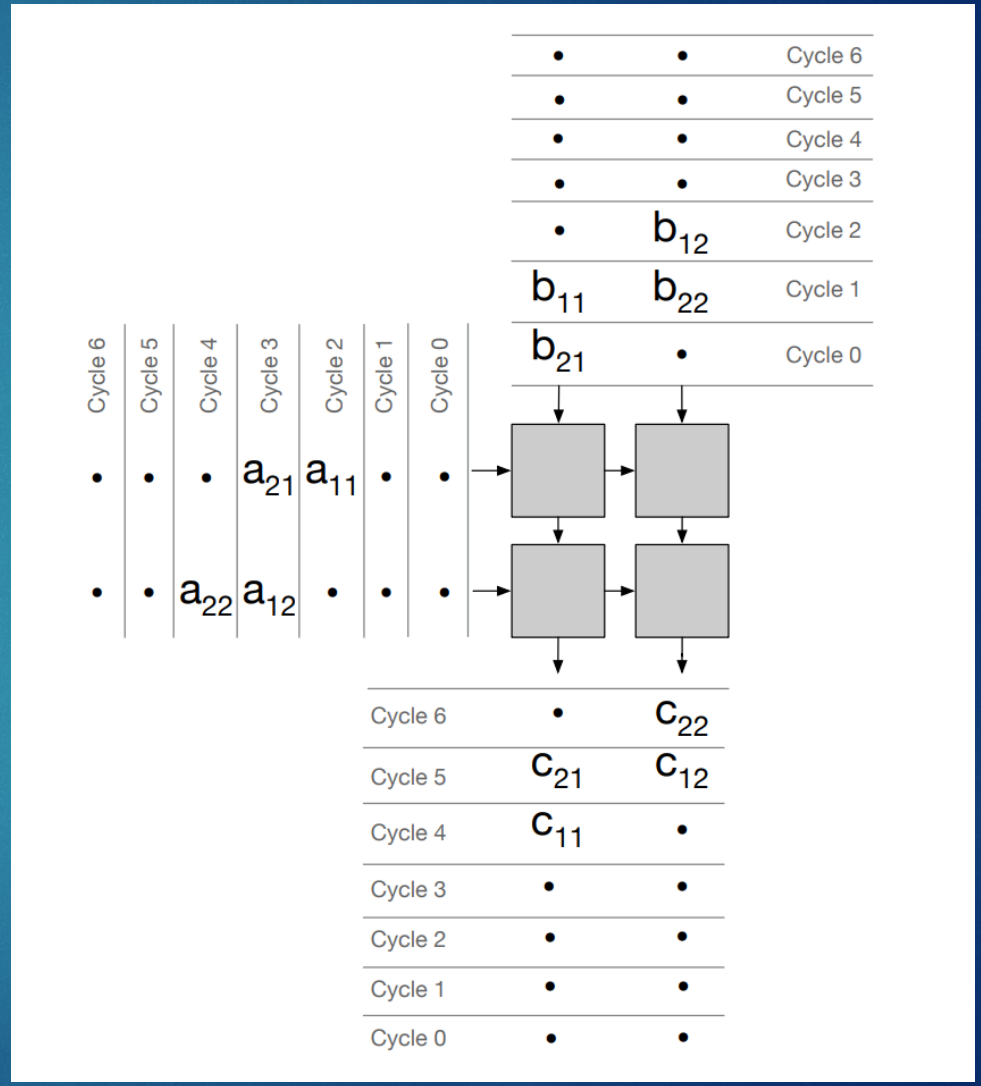
All rights reserved

2x2 Systolic Array – Preload Weight

In the above example, we assumed the B matrix is already loaded in the systolic array.

If we want to load B matrix to the array, we need additional 2 cycles of operation

In the example to the right, B matrix is first loaded in to the array (cycle 0 – 2), then the matrix multiplication is performed (cycle 2 – 6)



TetraMem Confidential

All rights reserved

2x2 Systolic Array with double buffered weight storage

The example above represents an major issue. The systolic array needs to sit idle while the B matrix is loading. If we want to load a new B matrix for each computation, the effective throughput is halved.

To overcome this, we need to **double-buffer** the B matrix storage element in each PE, ie: we need two registers to store B. One register to perform the matrix multiplication for the current cycle; and another register to load the B matrix for the next matrix multiplication.

The example on the right shows a double buffered systolic array performing two consecutive matrix multiply computation.

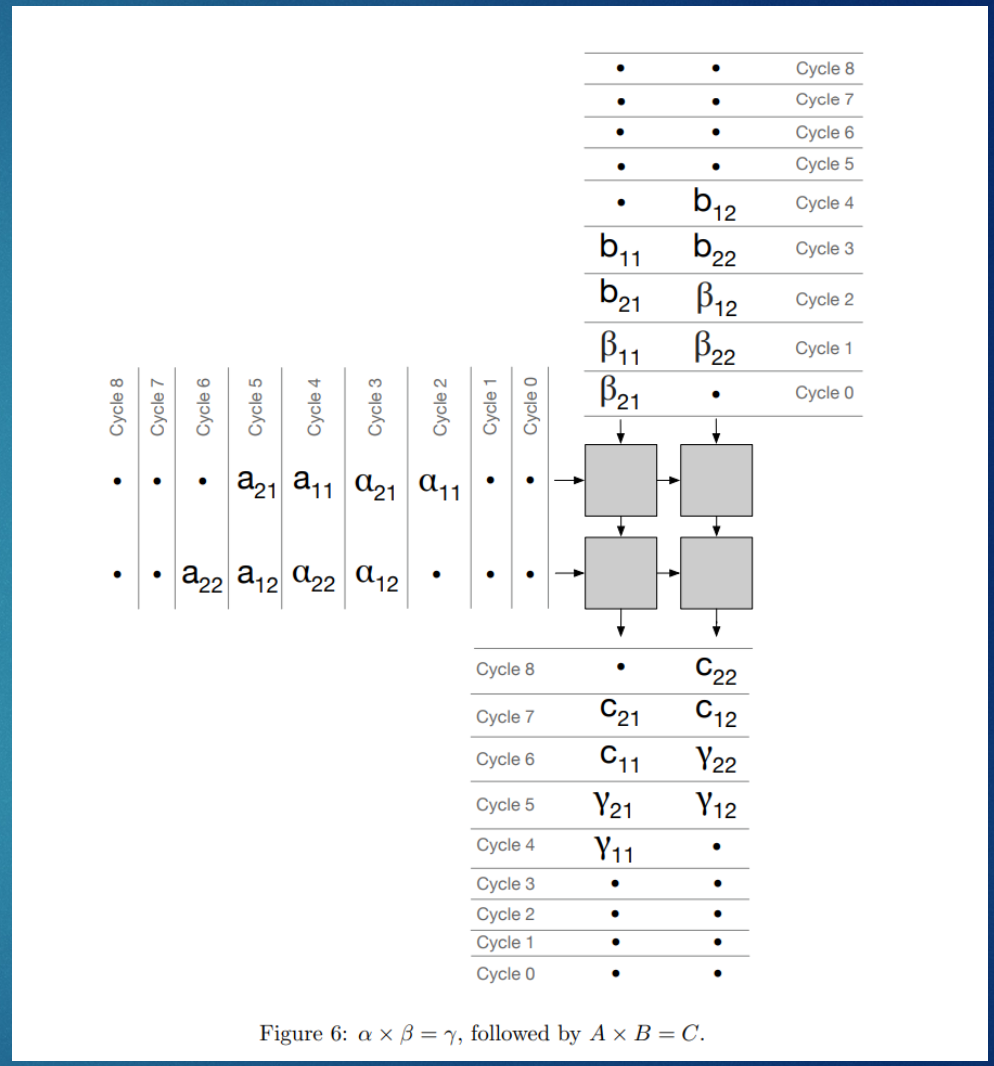


Figure 6: $\alpha \times \beta = \gamma$, followed by $A \times B = C$.

TetraMem Confidential

All rights reserved

NxN Systolic Array

Similarly, propagate weight buffer control signals for each column of PEs mesh, we can design a Systolic Array module with NxN PEs mesh to support the consecutive matrix multiplications for NxM and MxN matrixes with M computing clock cycle(Additional output latency caused by the input A and B, are not affect the computing throughput). As page 10 showing, the 2x2 and 2x2 matrix multiplications case using 2x2 PEs systolic array, the first C matrix output will be ready at 6th clock cycle and second C matrix output will be ready on the 6th + 2(computing cycle) = 8th clock cycle.

The NxN Systolic Array module interface is defined as following with Two parameters, DIN_WIDTH and N. All the elements of matrix are signed data.

```
Module systolic_array #(parameters DIN_WIDTH = 8, parameters N = 4 ) (
  input logic rst_n,                                  //active low reset
  input logic clk,                                    //clock
  input logic [2*DIN_WIDTH-1:0] c_din[0:N-1],         // The partial_sum input data of first row PEs. On top level, it can be connected to c_dout of last row PEs systolic_array
                                                      // With appropriate control logic for the loop back and propagate in each PEs of Systolic array. The Design can support
                                                      // the matrix multiplications for NxM and MxN matrix, This signal need be reset to be 0 when start the new matrix data

  input logic [DIN_WIDTH-1:0] a_din[0:N-1],           // N elements data of A matrix  ( column number ) , assuming each element is DIN_WIDTH bits Signed data
  input logic [DIN_WIDTH-1:0] b_din[0:N-1],           // N elements data of B matrix ( row number ) , assuming each element is DIN_WIDTH bits Signed data
  input logic in_valid,                               // Toggle to be high to indicates last elements data of A matrix and B matrix are ready (each M clock cycle
                                                      // depend on the M of  input matrix)

  output logic [2*DIN_WIDTH-1:0] c_dout[N-1:0],       // N (number of column of PEs mesh) Elements data of output C matrix, each element is 2*DIN_WIDTH bits
                                                      // Signed data, Default output is zero when data are not ready
output logic out_valid                                // Toggle to be high to indicates last elements data of C matrix output data are ready  (each M clock cycle
                                                      // depend on the M of  input matrix)

);
```

TetraMem Confidential

All rights reserved

Sub System Design with Systolic Array

Based on the Systolic Array module design descripted on page 11. The Sub system integrate the FIFOs, Systolic Array Controller, Data alignment unit as well as the instances of Systolic Array module to compute consecutive multiplication for matrixes (stream data).

In this sub system, data reading and writing are based on the sys_clk and the systolic Array operation are based on the sr_clk. A FIFOs are used to buffer the data between the bus and computing units. The width of bus data is one design parameter "BUS_WIDTH" which must be 2*DIN_WIDTH*N. The sys_clk/sr_clk can also be adjustable on the system level. Additional Control Signals "M_minus_one[7:0]" are also used to indicates the M - 1 for NxM and MxN matrix computing.

You can assume the DUT interface of the sub system level are as following :

```
Module sub_sys # (parameters DIN_WIDTH = 8, parameters N = 4, parameters BUS_WIDTH = 2*DIN_WIDTH*N) (
  input logic rst_n,                          //active low reset
  input logic sys_clk,                        //sys_clk is used for Data read and write
  input logic sr_clk,                         // sr_clk is used for the operation of Systolic Array and controller logic

  input logic[7:0] M_minus_one,               // Set this signal value to be "M-1" to indicates the NxM and MxN matrix multiplications

  input logic [BUS_WIDTH-1:0]  din,           // Input data for input FIFO. Each sample include N elements of Matrix A(one column) and N elements of matrix B(one row)
                                              // bits width of each matrix's element is DIN_WIDTH,
  input logic wr_fifo,                        // Write input FIFO signal, active high
  input logic rd_fifo,                        // Read output FIFO signal, active high
  output logic in_fifo_full,                  // Input data FIFO is full  (active high)
  output logic [BUS_WIDTH-1:0] dout,          // Read data of output FIFO, including N elements (number of column) of Matrix C output
  output logic out_fifo_empty                 // Output FIFO is empty, active high
);
```

TetraMem Confidential

All rights reserved

Your assignment

Please write the UVM testbench to verify the design for the both Systolic Array module level and sub system level.

Basic requirements of your DV testbench including:
1) Your testbench must include the necessary UVM components such as driver，monitor， scoreboard，sequencer;
2) Should be able to create the constrained random test data and check the DUT computing output automatically;
3) Can support to verify the different design parameters and clock relationship on sub system level;
4) Try your best to reuse the DV testbench components for module level and sub system level verification.

You don’t need implement the RTL design of DUT. But a golden reference module of matrix multiplications computing is necessary for automatic result checking in the scoreboard. A cycle accuracy performance/function model for the DUT using either system C or System Verilog will be a big plus, if your deliverables include it.

Your deliverables

Please package and submit the following to your interviewer:

- Source code of your DV testbench
- A short documentation of your verification plan and DV structure and clearly stating any assumptions or other optimizations you made to enhance the verification efficiency.
- If you have a Verilog simulator, then please run the simulation and submit the log files and some screenshot of the waveform of consecutive matrix multiplications for the both Systolic Array level and sub system level.
- If you do not have a simulator, you are encouraged to run simulation on the following website: https://edaplayground.com/

TetraMem Confidential

All rights reserved

Thanks for your time and good luck!

All rights reserved